

Design Patterns in Python

Design Patterns?

- Design patterns are reusable solutions to common software design problems.
- They provide proven approaches instead of reinventing solutions.
- Improve consistency, reliability, and maintainability.
- Describe both the problem and a general solution.
- Can be adapted to different implementations.

What Are Design Patterns?

- A design pattern is a model solution to a recurring problem.
- Focuses on software structure and behavior.
- Not a complete implementation.
- Serves as a blueprint for solving problems.
- Encourages best practices in software development.

Design Pattern Benefits

- Promote reusable solutions.
- Improve code readability.
- Reduce development time.
- Encourage maintainable designs.
- Provide a common vocabulary among developers.

History of Design Patterns

- Popularized by the "Gang of Four" (GoF) book in 1995.
- Authors: Gamma, Helm, Johnson, and Vlissides.
- First comprehensive software design pattern reference.

- Still considered an authoritative resource.
- Forms the basis of modern pattern-based design.

Design Pattern Classification

- Design patterns are divided into three categories.
- Creational patterns focus on object creation.
- Structural patterns focus on object composition.
- Behavioral patterns focus on object interaction.
- Each category solves a different design concern.

Creational Patterns

- Deal with object creation mechanisms.
- Hide creation complexity.
- Improve flexibility during object construction.
- Examples include Builder and Prototype.
- Promote loose coupling.

Structural Patterns

- Focus on class and object composition.
- Help organize relationships between objects.
- Simplify complex structures.
- Examples include Adapter and Proxy.
- Improve system flexibility.

Behavioral Patterns

- Define communication between objects.
- Manage responsibilities and interactions.
- Improve collaboration between components.

- Reduce tight coupling.
- Focus on runtime behavior.

SOLID Principles

- Five principles of object-oriented design.
- Improve maintainability and scalability.
- Encourage loosely coupled code.
- Form the foundation of many design patterns.
- Widely used in modern software development.

Single Responsibility Principle (SRP)

- A class should have only one reason to change.
- Focuses on a single responsibility.
- Simplifies maintenance.
- Reduces complexity.
- Improves testability.

```
class Invoice:
```

```
    def calculate_total(self):
```

```
        pass
```

```
class InvoicePrinter:
```

```
    def print_invoice(self):
```

```
        pass
```

Open/Closed Principle (OCP)

- Software entities should be open for extension.
- Closed for modification.
- New behavior should be added without changing existing code.
- Reduces risk of introducing bugs.

- Encourages abstraction.

```
class Shape:
```

```
    def area(self):
```

```
        pass
```

Liskov Substitution Principle (LSP)

- Derived classes must be replaceable for base classes.
- Subclasses should not break expectations.
- Ensures proper inheritance usage.
- Promotes reliable polymorphism.
- Prevents unexpected behavior.

```
class Bird:
```

```
    def fly(self):
```

```
        pass
```

Interface Segregation Principle (ISP)

- Clients should not depend on methods they don't use.
- Prefer small focused interfaces.
- Reduces unnecessary dependencies.
- Improves maintainability.
- Encourages modular design.

```
class Printer:
```

```
    def print(self):
```

```
        pass
```

Dependency Inversion Principle (DIP)

- High-level modules should depend on abstractions.
- Low-level modules should depend on abstractions.

- Reduces coupling.
- Improves flexibility.
- Supports dependency injection.

```
class Database:
```

```
    def save(self):
```

```
        pass
```

Interfaces in Python

- Interfaces define expected behavior.
- Python uses Abstract Base Classes (ABCs).
- Implemented through the abc module.
- Enforce required methods and properties.
- Support the Interface Segregation Principle.

Abstract Base Classes (ABC)

- Introduced through PEP 3119.
- Available since Python 2.6 and 3.0.
- Prevent direct instantiation.
- Define abstract methods.
- Act as contracts for subclasses.

```
from abc import ABC, abstractmethod
```

```
class Shape(ABC):
```

```
    @abstractmethod
```

```
    def area(self):
```

```
        pass
```

Concrete Class Implementation

- Inherits from an abstract base class.

- Implements all abstract methods.
- Can be instantiated normally.
- Must satisfy interface requirements.
- Python raises errors for missing implementations.

```
class Circle(Shape):
```

```
    def area(self):
```

```
        return 3.14 * 5 * 5
```

Builder Pattern

- Classification: Creational.
- Separates object construction from representation.
- Supports complex object creation.
- Encapsulates assembly logic.
- Client interacts only with the abstraction.

Builder Pattern Benefits

- Reduces constructor complexity.
- Avoids large parameter lists.
- Hides internal implementation details.
- Enforces object creation order.
- Supports multiple object representations.

Builder Pattern Structure

- Client creates a Director.
- Director controls construction steps.
- Builder defines construction methods.
- Concrete Builder implements those methods.
- Product is returned to the client.

```
class Computer:
    def __init__(self):
        self.cpu = None
        self.ram = None
```

Simple Builder Example:-

```
class ComputerBuilder:
    def __init__(self):
        self.computer = Computer()

    def add_cpu(self):
        self.computer.cpu = "Intel"

    def add_ram(self):
        self.computer.ram = "16GB"

    def build(self):
        return self.computer
```

Builder Pattern Advantages

- Separates "How" from "What".
- Encapsulates construction logic.
- Allows different implementations.
- Improves readability.
- Supports step-by-step assembly.

Prototype Pattern

- Classification: Creational.
- Creates new objects by cloning existing ones.
- Uses a prototype instance.
- Supports runtime object creation.
- Reduces the number of classes required.

Prototype Pattern Benefits

- Faster than building objects repeatedly.
- Simplifies object creation.
- Supports runtime inheritance.
- Allows object customization after cloning.
- Reduces duplication.

Shallow Copy

- Creates a new object.
- Copies references to nested objects.
- Faster than deep copying.
- Changes in nested objects may affect both copies.
- Useful for simple structures.

```
import copy
```

```
original = [1, [2, 3]]
```

```
clone = copy.copy(original)
```

Deep Copy

- Recursively copies nested objects.
- Produces independent copies.
- Uses `copy.deepcopy()`.
- Consumes more memory.
- Can be expensive for large structures.

```
import copy
```

```
original = [1, [2, 3]]
```

```
clone = copy.deepcopy(original)
```

Prototype Manager

- Stores reusable prototypes.
- Retrieves prototypes when needed.
- Centralizes cloning operations.
- Simplifies object management.
- Common in large systems.

Adapter Pattern

- Classification: Structural.
- Converts one interface into another.
- Allows incompatible classes to work together.
- Also called the Wrapper Pattern.
- Promotes code reuse.
- Existing code expects one interface.
- New classes expose different interfaces.
- Adapter bridges the gap.
- Avoids rewriting existing code.
- Supports Open/Closed Principle.

```
class Vendor:
```

```
    def get_address(self):
```

```
        return "123 Main St"
```

Object Adapter

- Uses composition.
- Wraps the adaptee object.
- Delegates calls to the adaptee.
- Works with all adaptee subclasses.
- Preferred over inheritance.

```

class VendorAdapter:

    def __init__(self, vendor):

        self.vendor = vendor

    @property

    def address(self):

        return self.vendor.get_address()

```

Class Adapter

- Uses inheritance.
- Extends the adaptee class.
- Overrides methods when needed.
- Works with one adaptee type.
- Less flexible than object adapters.

```

class VendorAdapter(Vendor):

    @property

    def address(self):

        return self.get_address()

```

Adapter Pattern Advantages

- Reuses existing code.
- Integrates third-party libraries.
- Reduces duplicate code.
- Supports unforeseen interfaces.
- Encourages abstraction.

Proxy Pattern

- Classification: Structural.
- Controls access to another object.

- Acts as a placeholder or representative.
- Exposes the same interface as the real object.
- Adds behavior without modifying the original object.

Proxy Pattern Types

- Protection Proxy controls access permissions.
- Virtual Proxy delays object creation.
- Remote Proxy hides communication details.
- Smart Proxy adds housekeeping operations.
- Frequently used in enterprise applications.

Protection Proxy

- Restricts access to sensitive data.
- Verifies permissions before execution.
- Improves security.
- Common in business systems.
- Supports access control requirements.

```
class Employee:
```

```
    def get_salary(self):
```

```
        return 100000
```

Proxy Example

```
class EmployeeProxy:
```

```
    def __init__(self, employee, is_admin):
```

```
        self.employee = employee
```

```
        self.is_admin = is_admin
```

```
    def get_salary(self):
```

```
        if self.is_admin:
```

```
            return self.employee.get_salary()
```

```
return "Access Denied"
```

Virtual Proxy

- Delays expensive object creation.
- Creates objects only when needed.
- Improves startup performance.
- Conserves resources.
- Similar to lazy loading.

```
from functools import lru_cache
```

```
@lru_cache()
```

```
def load_data():
```

```
    return "Large Dataset"
```

Proxy Pattern Advantages

- Controls access transparently.
- Follows Open/Closed Principle.
- Adds functionality without changing the subject.
- Supports security and optimization.
- Can combine multiple proxy types.

Strategy Pattern

- Classification: Behavioral Design Pattern.
- Defines a family of algorithms.
- Encapsulates each algorithm separately.
- Makes algorithms interchangeable.
- Also known as the Policy Pattern.

Purpose of the Strategy Pattern

- Separate algorithms from business logic.
- Allow algorithms to vary independently.
- Eliminate large if/elif/else blocks.
- Improve maintainability and flexibility.
- Support future extensions without modifying existing code.

Motivating Example: Shipping Cost Calculator

- Application calculates shipping costs.
- Must support Federal Express, UPS, and Postal Service.
- Each shipping provider has a different algorithm.
- New shipping providers may be added later.
- System should remain easy to extend.

Problems with Traditional Design

- Shipping logic mixed with order logic.
- Violates Single Responsibility Principle.
- Violates Open/Closed Principle.
- Violates Dependency Inversion Principle.
- Requires long if/elif/else statements.

Bad Example

```
class Order:
    def calculate_shipping(self, shipper):
        if shipper == "UPS":
            return 10
        elif shipper == "FedEx":
            return 15
        elif shipper == "Postal":
```

```
return 5
```

Problems with if/elif/else Chains

- Difficult to maintain.
- Hard to test individual algorithms.
- Every new algorithm requires modifying existing code.
- Increases coupling.
- Considered a design smell.

Strategy Pattern Structure

- Context contains business logic.
- Strategy interface defines the algorithm contract.
- Concrete Strategies implement different algorithms.
- Context delegates work to a strategy.
- Client chooses the desired strategy.

Strategy Pattern Components

- Context: Uses a strategy object.
- Strategy (ABC): Defines the common interface.
- Concrete Strategy: Implements a specific algorithm.
- Client: Selects and injects a strategy.
- Algorithms remain independent.

Strategy Interface

- Created using Abstract Base Classes (ABC).
- Defines required methods.
- All strategies follow the same contract.
- Supports polymorphism.

- Encourages programming to an interface.

```
from abc import ABC, abstractmethod
```

```
class ShippingStrategy(ABC):
```

```
    @abstractmethod
```

```
    def calculate(self, weight):
```

```
        pass
```

Concrete Strategy: UPS

- Implements UPS shipping logic.
- Encapsulates UPS-specific calculations.
- Independent of other strategies.
- Can be tested separately.
- Easy to replace.

```
class UPSStrategy(ShippingStrategy):
```

```
    def calculate(self, weight):
```

```
        return weight * 2
```

Concrete Strategy: FedEx

- Implements FedEx shipping rules.
- Uses the same interface.
- Interchangeable with other strategies.
- Requires no changes to the Context.
- Supports extension.

```
class FedExStrategy(ShippingStrategy):
```

```
    def calculate(self, weight):
```

```
        return weight * 3
```

Concrete Strategy: Postal Service

- Represents postal shipping calculations.
- Follows the common strategy contract.
- Easily replaceable.
- Isolated from other implementations.
- Supports independent testing.

```
class PostalStrategy(ShippingStrategy):
```

```
    def calculate(self, weight):
```

```
        return weight * 1.5
```

Context Class

- Contains core business functionality.
- Does not know algorithm details.
- Uses the strategy interface.
- Delegates calculations to strategies.
- Remains closed for modification.

```
class Order:
```

```
    def __init__(self, strategy):
```

```
        self.strategy = strategy
```

```
    def shipping_cost(self, weight):
```

```
        return self.strategy.calculate(weight)
```

Using the Strategy Pattern

- Client chooses the desired strategy.
- Strategy is injected into the context.
- Context executes the algorithm.
- Algorithms can be switched dynamically.
- No code modification required.

```
order = Order(UPSStrategy())
```

```
print(order.shipping_cost(10))
```

How Strategy Fixes Design Problems

- Removes shipping concerns from orders.
- Separates each algorithm.
- Keeps existing code unchanged.
- Allows unlimited extensions.
- Supports Dependency Inversion.

Single Responsibility Principle

- Order class handles orders only.
- Shipping algorithms handle shipping only.
- Responsibilities are clearly separated.
- Easier maintenance.
- Cleaner architecture.

Open/Closed Principle

- Existing classes remain unchanged.
- New strategies can be added easily.
- Supports extension without modification.
- Reduces bug risk.
- Improves scalability.

```
class DHLStrategy(ShippingStrategy):
```

```
    def calculate(self, weight):
```

```
        return weight * 4
```

Dependency Inversion Principle

- Context depends on abstractions.
- Context does not depend on concrete strategies.
- Reduces coupling.
- Increases flexibility.
- Simplifies testing.

Testing Benefits

- Test each algorithm independently.
- Mock strategies can be injected.
- Predictable test behavior.
- Easier unit testing.
- Improved code quality.

```
class MockStrategy(ShippingStrategy):
```

```
    def calculate(self, weight):
```

```
        return 109
```

Advantages of Strategy Pattern

- Eliminates complex conditional logic.
- Supports runtime algorithm switching.
- Improves maintainability.
- Encourages clean architecture.
- Follows SOLID principles.

Strategies as Functions

- Separate classes are not always required.
- Functions can act as strategies.
- Simpler implementation.

- Useful for lightweight algorithms.
- Reduces boilerplate code.

```
def ups_shipping(weight):
```

```
    return weight * 2
```

Using Function Strategies

```
class Order:
```

```
    def __init__(self, strategy):
```

```
        self.strategy = strategy
```

```
    def shipping_cost(self, weight):
```

```
        return self.strategy(weight)
```

```
order = Order(ups_shipping)
```

Strategies as Lambda Expressions

- Useful for simple calculations.
- Extremely concise.
- Avoids creating extra functions.
- Suitable for small algorithms.
- Less suitable for complex logic.

```
ups_strategy = lambda weight: weight * 2
```

```
print(ups_strategy(10))
```

Different Strategy Implementations

- Class per algorithm.
- Standalone functions.
- Lambda expressions.
- Dynamically loaded strategies.
- Dependency injection frameworks.

When to Use Strategy Pattern

- Multiple algorithms solve the same problem.
- Algorithms change frequently.
- Large conditional statements exist.
- Runtime algorithm selection is needed.
- SOLID compliance is important.

When to Avoid Strategy Pattern

- Only one algorithm exists.
- Algorithms rarely change.
- Additional abstraction adds unnecessary complexity.
- Very small projects.
- Simpler solutions are sufficient.

Red Flags Suggesting Strategy Pattern

- Long if/elif/else chains.
- Repeated algorithm selection logic.
- Frequent code modifications for new behaviors.
- Tight coupling between algorithms and business logic.
- Difficulty testing individual algorithms.